

Kalman & Bayesian Filters - Walkthrough

Course: AI for Robotics **Submitted by:** Aleem Ul Hassan **Date:** June 17, 2026 **Deliverable:** [Kalman_Filter_Walkthrough.ipynb](#) (runs end to end in Google Colab)

1. What this code is about

The notebook is a guided build-up of the **Kalman filter**, the standard recursive estimator used in robotics to track a system's true state from noisy sensor data. It follows the well-known *Kalman and Bayesian Filters in Python* progression and ends with a working position + velocity tracker.

The core idea: a robot never knows its exact state. Both its motion and its sensors are noisy. The Kalman filter represents what we believe as a **Gaussian** (a mean plus a variance) and updates that belief in two repeating steps:

- 1 **Predict** - move the belief forward using the motion model. Uncertainty grows.
- 2 **Update (correct)** - fuse a new sensor measurement into the belief. Uncertainty shrinks.

The filter weights the prediction against the measurement by how much it trusts each, using the **Kalman gain**. For linear systems with Gaussian noise it is the provably optimal estimator.

2. Section-by-section breakdown

Section	What it demonstrates
0. Setup	Installs <code>filterpy</code> , imports numpy / matplotlib / scipy
1. Descriptive statistics	mean, median, variance, std; variance = std squared
2. The Gaussian	the normal PDF, its CDF (probability over a range), and that small variance = high confidence
3. Noisy measurements	one reading is unreliable; the average of 500 converges to the truth
4. The 1-D Kalman filter	<code>predict</code> adds Gaussians (variance grows); <code>update</code> multiplies Gaussians (variance shrinks)
5. Tracking in 1-D	a simulated "dog" moving at constant velocity, filtered over 10 steps
5.1 Kalman gain	the scalar K that balances trust between prediction and measurement

Section	What it demonstrates
6. Multivariate filter	state becomes a vector [position, velocity]; variance becomes a covariance matrix; covariance ellipse; process-noise matrix Q
7. Full filter	<code>filterpy.KalmanFilter</code> constant-velocity tracker run on 50 simulated steps, plus the belief sharpening over time

3. The math in one place

1-D predict (adding two independent Gaussians):

```
mu = mu + mu_move
var = var + var_move
```

1-D update (multiplying two Gaussians):

```
mu = (var1*mu2 + var2*mu1) / (var1 + var2)
var = (var1*var2) / (var1 + var2)
```

Kalman gain form of the update:

```
y = z - x          # residual (innovation)
K = P / (P + R)    # Kalman gain, 0..1
x = x + K*y        # corrected mean
P = (1 - K) * P    # corrected variance
```

Multivariate (matrix form, used in section 6-7):

```
Predict: x = F x + B u          P = F P F^T + Q
Update:  uses measurement z, noise R, and measurement matrix H
```

K near 1 means "trust the sensor", K near 0 means "trust the prediction".

4. Bugs found and fixed

The original pasted snippets were transcribed from the textbook/slides and would **not run** in Colab. The problems fell into two groups.

4.1 Code-level bugs (would crash)

#	Original	Problem	Fix
1	<code>np.var(X), np.std(X)</code>	list was defined as lowercase <code>x</code> ; <code>X</code> is undefined → <code>NameError</code>	use <code>x</code> consistently
2	<code>print(...)</code> <code>print(...)</code> on one line	two statements, no separator → <code>SyntaxError</code>	split onto separate lines
3	<code>likelihood (z, sensor_var)</code>	missing <code>=</code> assignment in the filter loop	<code>likelihood = (z, sensor_var)</code>
4	<code>Q = e</code>	<code>e</code> is undefined; should be zero process noise	<code>Q = 0.0</code>
5	<code>plot_gaussian_pdf(..., xlim(4, 16), ...)</code>	<code>xlim(4, 16)</code> called as a function instead of a keyword arg → <code>TypeError/SyntaxError</code>	<code>xlim=(4, 16)</code>
6	<code>run(xe=(0,0.)) ...</code> <code>kf = pos_vel_filter(x@, ...)</code>	parameter named <code>xe</code> but used as <code>x0</code> ; <code>x@</code> is a transcription artifact	renamed consistently to <code>x0</code>

4.2 Environment bugs (the real blocker)

Every original snippet imported book-only modules and a custom plotting context:

```
from book_format import set_figsize, figsize
import code.book_plots as bp
import code.kf_internal as kf_internal
from code.kf_internal import DogSimulation
import code.mkf_internal as ...
from code.nonlinear_plots import plot_gaussians
with interactive_plot(): ...
```

None of these exist in Colab - they ship only inside the book's GitHub repo. Importing them raises `ModuleNotFoundError` immediately, so the original code cannot run as pasted regardless of the small syntax fixes above.

Fix: every helper was re-implemented inline so the notebook is fully self-contained:

- `gaussian`, `plot_gaussian_pdf`, `norm_cdf` - defined directly (CDF uses `scipy.stats.norm`).
- `DogSimulation`, `print_gh` - small equivalent classes/functions.
- `plot_covariance_ellipse` - written with `matplotlib.patches.Ellipse`.
- `interactive_plot()` context removed; plain `plt.figure()` / `plt.show()` used.
- The only external install is `filterpy` (`!pip install filterpy -q`), used for

`KalmanFilter`, `predict/update`, and `Q_discrete_white_noise`.

5. Verification

The notebook was executed end to end with `jupyter nbconvert --execute`. All 18 code cells ran with zero errors. Sample results:

- Process-noise matrix `Q_discrete_white_noise(dim=2, dt=1, var=2.35) = [[0.5875, 1.175], [1.175, 2.35]]`.
- Constant-velocity tracker final estimate after 50 steps: position ~48.5 m, velocity ~9.2 (tracking the true accelerating-ish path).
- The "belief sharpening" plot shows the Gaussian getting taller and narrower over time - uncertainty

shrinking exactly as theory predicts.

6. How to run

- 1 Open Google Colab → File → Upload notebook → `Kalman_Filter_Walkthrough.ipynb`.
- 2 Run cell 0 first (installs `filterpy`).
- 3 Run all remaining cells top to bottom (Runtime → Run all). Every figure regenerates inline.

No other files are needed.

7. Key takeaway

A Kalman filter is just **"predict, then correct"** repeated forever on a Gaussian belief. Prediction uses the motion model and adds uncertainty; correction fuses a measurement and removes uncertainty; the Kalman gain decides the balance. That single loop is what lets a robot estimate where it really is from imperfect sensors.